Design and Implementation of a Classical Caesar Cipher Simulator in Intel 8086 Microsoft Macro Assembler (MASM) Architecture

Abstract and Executive Summary

This system laboratory report presents the architectural design, mathematical formulation, and low-level assembly language implementation of a classical Caesar Cipher cryptographic simulator developed for the Intel 8086 16-bit microprocessor environment using the Microsoft Macro Assembler (`MASM`). The Caesar Cipher represents one of the earliest and most foundational symmetric-key substitution cryptosystems in the history of information security. Operating as a monoalphabetic shift cipher, the algorithm transforms each alphabetic character in a plaintext message by displacing its ordinal position along the Latin alphabet by a fixed integer offset—in this system specification, a shift key of $K = +3$. When the shift operation advances a character beyond the terminal boundary of the alphabet (`Z` for uppercase or `z` for lowercase), modular arithmetic wraps the displacement back to the initial boundary (`A` or `a`), ensuring a bijective mapping across the 26-character domain.

While high-level programming languages abstract character manipulation behind built-in string libraries and automatic type management, implementing cryptographic primitives directly in 16-bit real-mode assembly exposes the fundamental interaction between processor registers, flag transitions, memory segmentation, and operating system interrupt vectors. The developed system (`CAESAR.ASM`) fulfills three core operational requirements:

1. Interactive acquisition of a variable-length plaintext string from standard input utilizing MS-DOS Interrupt 21h buffered keyboard services (`AH = 0Ah`).
2. Forward cryptographic transformation (encryption) applying a $+3$ modular displacement across both uppercase (`41h`–`5Ah`) and lowercase (`61h`–`7Ah`) ASCII ranges while preserving non-alphabetic symbols, whitespace, and punctuation.
3. Inverse cryptographic transformation (decryption) applying a -3 modular displacement to the generated ciphertext to deterministically reconstruct the original plaintext string, accompanied by a real-time, cycle-by-cycle hexadecimal and character trace rendered directly to the console.

System Attribute	Technical Specification	Architectural Role
Target Processor	Intel 8086 / 8088 (16-Bit Real Mode)	Executes byte-level ALU shifts, comparisons, and indexed memory addressing
Assembler Toolchain	Microsoft Macro Assembler (`MASM`) / `TASM` / `EMU8086`	Translates `.MODEL SMALL` mnemonics into relocatable 16-bit object code
Operating Environment	MS-DOS / `DOSBox` Virtual Machine	Provides character I/O and process termination via `INT 21h` services
Cryptographic Primitive	Monoalphabetic Caesar Cipher ($K = 3$)	Applies symmetric forward ($+3$) and reverse (-3) modular substitution
Character Encoding	7-Bit / 8-Bit Standard ASCII	Maps uppercase (`41h`–`5Ah`) and lowercase (`61h`–`7Ah`) contiguous ranges
Memory Model	`.MODEL SMALL` (Single `DS`, Single `CS`, `100h` Stack)	Optimizes data access via 16-bit near pointers (`SI`, `DI`, `BX`)

Theoretical Foundations of the Caesar Cipher and Modular Arithmetic

Historical Evolution and Cryptographic Taxonomy

Recorded by the Roman historian Suetonius in *De Vita Caesarum*, Julius Caesar employed a shift cipher with a fixed displacement of three positions to protect military correspondence of strategic significance (Caesar cipher - Wikipedia). Within modern cryptographic taxonomy, the Caesar Cipher is classified as a symmetric, monoalphabetic substitution cipher (The Caesar Cipher, Explained - Splunk). It is symmetric because the identical secret parameter—the shift key K—governs both the encryption and decryption transformations, differing only in the algebraic sign of the displacement. It is monoalphabetic because a given plaintext symbol is invariably mapped to the exact same ciphertext symbol throughout the entirety of a message, regardless of its positional index.

Mathematical Formulation in Modular Arithmetic

To formalize the Caesar Cipher algebraically, the 26 letters of the English Latin alphabet are bijectively mapped onto the ring of integers modulo 26, denoted as $\mathbb{Z}_{26} = \{0, 1, 2, \ldots, 25\}$ (How does the Caesar Cipher utilize the concept of modular arithmetic - EITCA). Under this canonical correspondence, A corresponds to 0, B to 1, C to 2, continuing sequentially through Z at 25.

Let $P \in \mathbb{Z}_{26}$ represent the zero-indexed ordinal value of a plaintext character, and let $K \in \mathbb{Z}_{26}$ represent the integer shift key (where $K = 3$ for the classical Caesar specification). The encryption function $E_K : \mathbb{Z}_{26} \to \mathbb{Z}_{26}$ computes the ciphertext ordinal C as:

$$C = E_K(P) = (P + K) \bmod 26$$

For the specific case of $K = 3$:

$$C = E_3(P) = (P + 3) \bmod 26$$

Conversely, the decryption function $D_K : \mathbb{Z}_{26} \to \mathbb{Z}_{26}$ is the algebraic inverse of E_K, recovering the original plaintext ordinal P from the ciphertext ordinal C by subtracting the shift key modulo 26:

$$P = D_K(C) = (C - K) \bmod 26$$

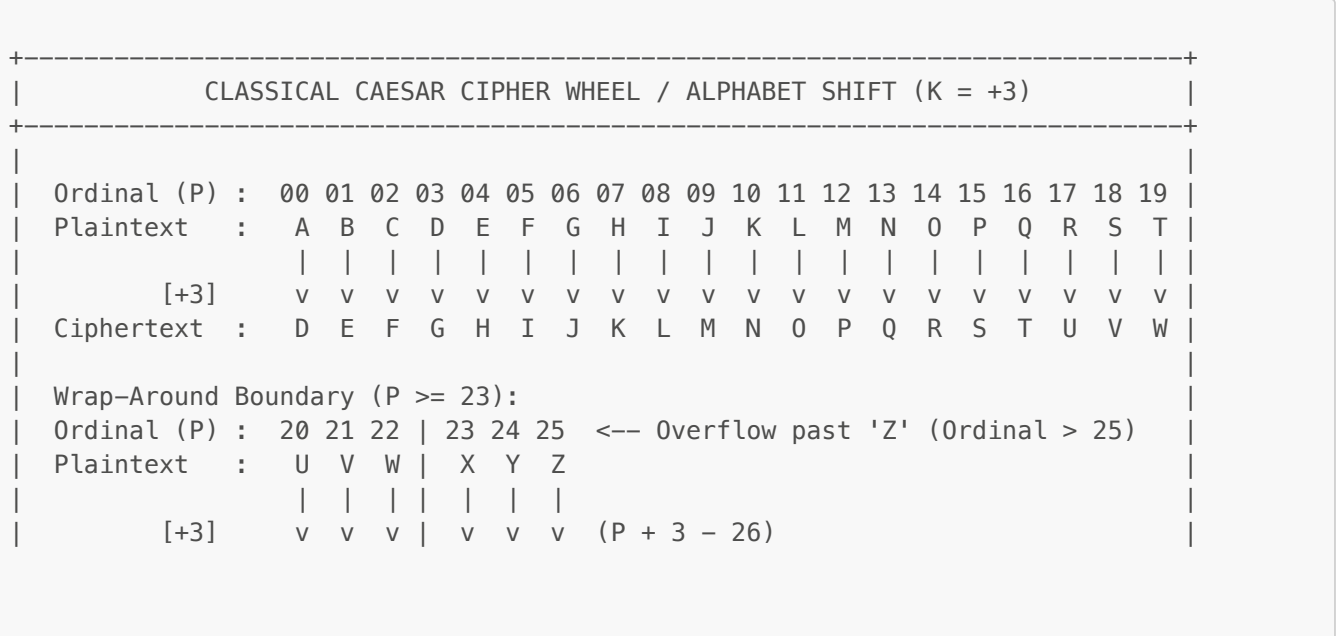
For $K = 3$, this inverse mapping is expressed as:

$$P = D_3(C) = (C - 3) \bmod 26 \equiv (C + 23) \bmod 26$$

Because $\mathbb{Z}_{26}$ forms an abelian group under addition modulo 26, every shift key K possesses a unique additive inverse $(-K \bmod 26 = 26 - K)$. Consequently, applying $D_K(E_K(P))$ yields:

$$D_K(E_K(P)) = ((P + K) - K) \bmod 26 = P \bmod 26 = P$$

This identity guarantees lossless, deterministic reconstruction of any input plaintext across the alphabetic domain.



```
|  Ciphertext  :   X  Y  Z |  A  B  C  <-- Wrapped to start of Alphabet       |
+-------------------------------------------------------------------------+
```

Translation from Zero-Indexed Ring $\mathbb{Z}_{26}$ to ASCII Byte Representation

In microprocessor hardware, characters are not stored as zero-indexed elements of $\mathbb{Z}_{26}$, but rather as 8-bit unsigned integers conforming to the ASCII standard. In the ASCII encoding table:

- Uppercase Latin letters (A through Z) occupy contiguous hexadecimal byte values 41h (65_{10}) through 5Ah (90_{10}).
- Lowercase Latin letters (a through z) occupy contiguous hexadecimal byte values 61h (97_{10}) through 7Ah (122_{10}).
- Numerals (0–9 at 30h–39h), punctuation, and control codes reside outside these two intervals.

Notice that between uppercase Z (5Ah, 90_{10}) and lowercase a (61h, 97_{10}), there is a 6-byte gap (5Bh through 60h) occupied by [, \,], ^, _, and `. If a program naively applied modular division (DIV) on raw ASCII bytes, characters would corrupt across case boundaries or require expensive 16-bit division instructions (DIV on the 8086 consumes 80–90 clock cycles per execution).

Instead, in 8086 assembly, the modular reduction $(P + K) \bmod 26$ is transformed into a conditional boundary comparison and subtraction/addition of 26_{10} (1Ah), executing in a fraction of the clock cycles:

1. **Uppercase Encryption** ($\text{Base} = \text{'A'} = 41\text{h}$, $\text{Upper Bound} = \text{'Z'} = 5\text{Ah}$):

$$b' = \begin{cases} b + 3 & \text{if } 41\text{h} \le b + 3 \le 5\text{Ah} \\ (b + 3) - 26 & \text{if } b + 3 > 5\text{Ah} \end{cases}$$

2. **Lowercase Encryption** ($\text{Base} = \text{'a'} = 61\text{h}$, $\text{Upper Bound} = \text{'z'} = 7\text{Ah}$):

$$b' = \begin{cases} b + 3 & \text{if } 61\text{h} \le b + 3 \le 7\text{Ah} \\ (b + 3) - 26 & \text{if } b + 3 > 7\text{Ah} \end{cases}$$

3. **Uppercase Decryption** ($\text{Lower Bound} = \text{'A'} = 41\text{h}$):

$$b = \begin{cases} b' - 3 & \text{if } b' - 3 \ge 41\text{h} \\ (b' - 3) + 26 & \text{if } b' - 3 < 41\text{h} \end{cases}$$

4. **Lowercase Decryption** ($\text{Lower Bound} = \text{'a'} = 61\text{h}$):

$$b = \begin{cases} b' - 3 & \text{if } b' - 3 \ge 61\text{h} \\ (b' - 3) + 26 & \text{if } b' - 3 < 61\text{h} \end{cases}$$

```
+-------------------------------------------------------------------------+
|            ASCII MEMORY MAP & BOUNDARY WRAP-AROUND ARITHMETIC           |
+-------------------------------------------------------------------------+
|                                                                         |
|  00h .. 40h : Control Chars, Space (20h), Digits (30h-39h), Symbols     |
|               --> [BYPASSED: Non-Alphabetic, Preserved Unchanged]       |
|                                                                         |
|  41h ('A') ---------------------------------------------------+         |
|    |        UPPERCASE ALPHABETIC DOMAIN (41h to 5Ah / 65 to 90)  |       |
|    |        * Encryption: AL = AL + 3                            |       |
|    |                      If AL > 5Ah ('Z') -> AL = AL - 1Ah (26)|       |
|    |        * Decryption: AL = AL - 3                            |       |
|    |                      If AL < 41h ('A') -> AL = AL + 1Ah (26)|       |
```

```
|  5Ah ('Z') ------------------------------------------------------+          |
|                                                                   |          |
|  5Bh .. 60h : Brackets & Symbols ('[', '\', ']', '^', '_', '`')   |          |
|               --> [BYPASSED: Gap Between Cases, Preserved Unchanged]        |
|                                                                   |          |
|  61h ('a') ------------------------------------------------------+          |
|    |        LOWERCASE ALPHABETIC DOMAIN (61h to 7Ah / 97 to 122)  |          |
|    |        * Encryption: AL = AL + 3                             |          |
|    |                      If AL > 7Ah ('z') -> AL = AL - 1Ah (26)|          |
|    |        * Decryption: AL = AL - 3                             |          |
|    |                      If AL < 61h ('a') -> AL = AL + 1Ah (26)|          |
|  7Ah ('z') ------------------------------------------------------+          |
|                                                                   |          |
|  7Bh .. FFh : Braces, Tilde, Extended ASCII                       |          |
|               --> [BYPASSED: Non-Alphabetic, Preserved Unchanged]           |
+-------------------------------------------------------------------------+
```

Intel 8086 Microprocessor Architecture and MASM Environment

Bus Interface Unit (BIU) and Execution Unit (EU)

The Intel 8086 microprocessor partitions internal processing across two asynchronous functional units: the Bus Interface Unit (`BIU`) and the Execution Unit (`EU`). The `BIU` manages physical 20-bit address generation ($1\ \text{MB}$ address space), instruction prefetching into a 6-byte queue, and bus cycles over the 16-bit multiplexed data/address bus. The `EU` contains the 16-bit Arithmetic Logic Unit (`ALU`), the 16-bit `FLAGS` register, and the general-purpose register file (`AX`, `BX`, `CX`, `DX`, `SI`, `DI`, `BP`, `SP`).

During the execution of `CAESAR.ASM`, the `BIU` computes physical memory addresses by shifting the 16-bit Data Segment (`DS`) register left by 4 bits (multiplying by 16_{10} or `10h`) and adding the 16-bit effective offset held in an index or base register (`SI` or `BX`):

$$\text{Physical Address} = (\text{DS} \times 10\text{h}) + \text{Offset}$$

Register Allocation Strategy in `CAESAR.ASM`

Efficient assembly programming requires disciplined register allocation so that inner loop operations avoid unnecessary memory spills while subroutines preserve calling context via stack frames (`PUSH`/`POP`).

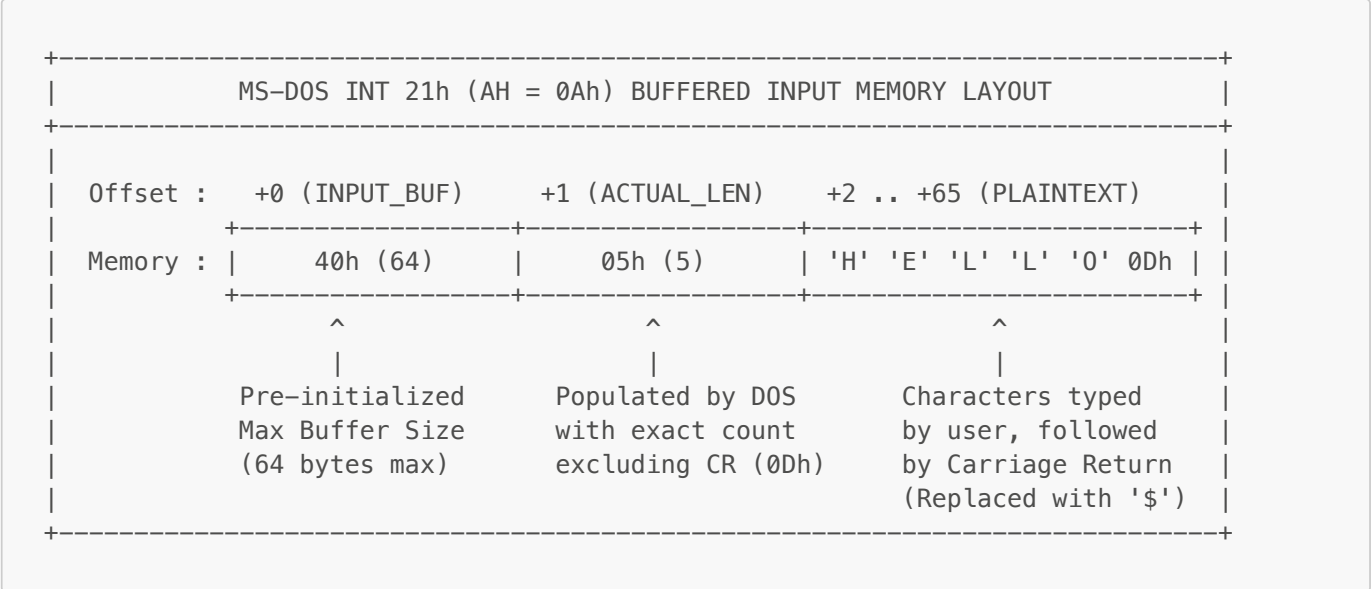
8086 Register	8-Bit Split	Assigned Role in `CAESAR.ASM`	Architectural Justification
`AX`	`AH` / `AL`	Primary Accumulator & DOS Function Selector	`AL` holds the active character undergoing cipher arithmetic and `XLAT` table lookup; `AH` selects the `INT 21h` service (`09h`, `0Ah`, `02h`, `4Ch`).
`BX`	`BH` / `BL`	Trace Cache & Base Register	`BL` caches the pre-transformation byte and `BH` caches the post-transformation byte for `PRINT_TRACE_ROW`; `BX` also serves as base pointer for `XLAT` and buffer termination (`PLAINTEXT[BX]`).
`CX`	`CH` / `CL`	Loop Counter	`CL` loads `ACTUAL_LEN` (number of characters entered), driving hardware `LOOP ENCRYPT_LOOP` and `LOOP DECRYPT_LOOP` instructions.
`DX`	`DH` / `DL`	I/O Address & Character Output Register	`DX` holds 16-bit effective offsets (`LEA DX, ...`) for DOS string I/O (`AH=09h`, `AH=0Ah`); `DL` holds single characters for `AH=02h`.

8086 Register	8-Bit Split	Assigned Role in `CAESAR.ASM`	Architectural Justification
`SI`	—	Source Index (Character Array Pointer)	Indexes sequentially (`0` to `N-1`) through `PLAINTEXT[SI]` and `ENCRYPTED_STR[SI]` using indexed addressing mode.
`DI`	—	Destination Index	Used with `ES:DI` and `REP MOVSB` during default string initialization if empty input is detected.
`DS` / **`ES`**	—	Data & Extra Segment Registers	Both pointed to `@DATA` under `.MODEL SMALL` to allow seamless indexed addressing and hardware string transfers (`MOVSB`).
`FLAGS`	—	Conditional Branching Status	`CF` (Carry Flag) and `ZF` (Zero Flag) are evaluated after `CMP` instructions by unsigned conditional jumps (`JB`, `JBE`, `JA`, `JAE`, `JNE`).

Memory Segmentation and DOS Function `0Ah` Buffer Mechanics

The program declares `.MODEL SMALL`, which instructs Microsoft Macro Assembler to combine all data declarations into a single 64 KB Data Segment (`_DATA`) and all executable instructions into a single 64 KB Code Segment (`_TEXT`), accompanied by a 256-byte (`.STACK 100H`) stack segment.

To read a user string safely without buffer overflow, `CAESAR.ASM` invokes MS-DOS Interrupt 21h with `AH = 0Ah` (Buffered Keyboard Input) (Stack Overflow - Interrupt 21, AH=0ah). Unlike single-character input (`AH = 01h`), Function `0Ah` requires `DS:DX` to point to a structured 3-part memory block:

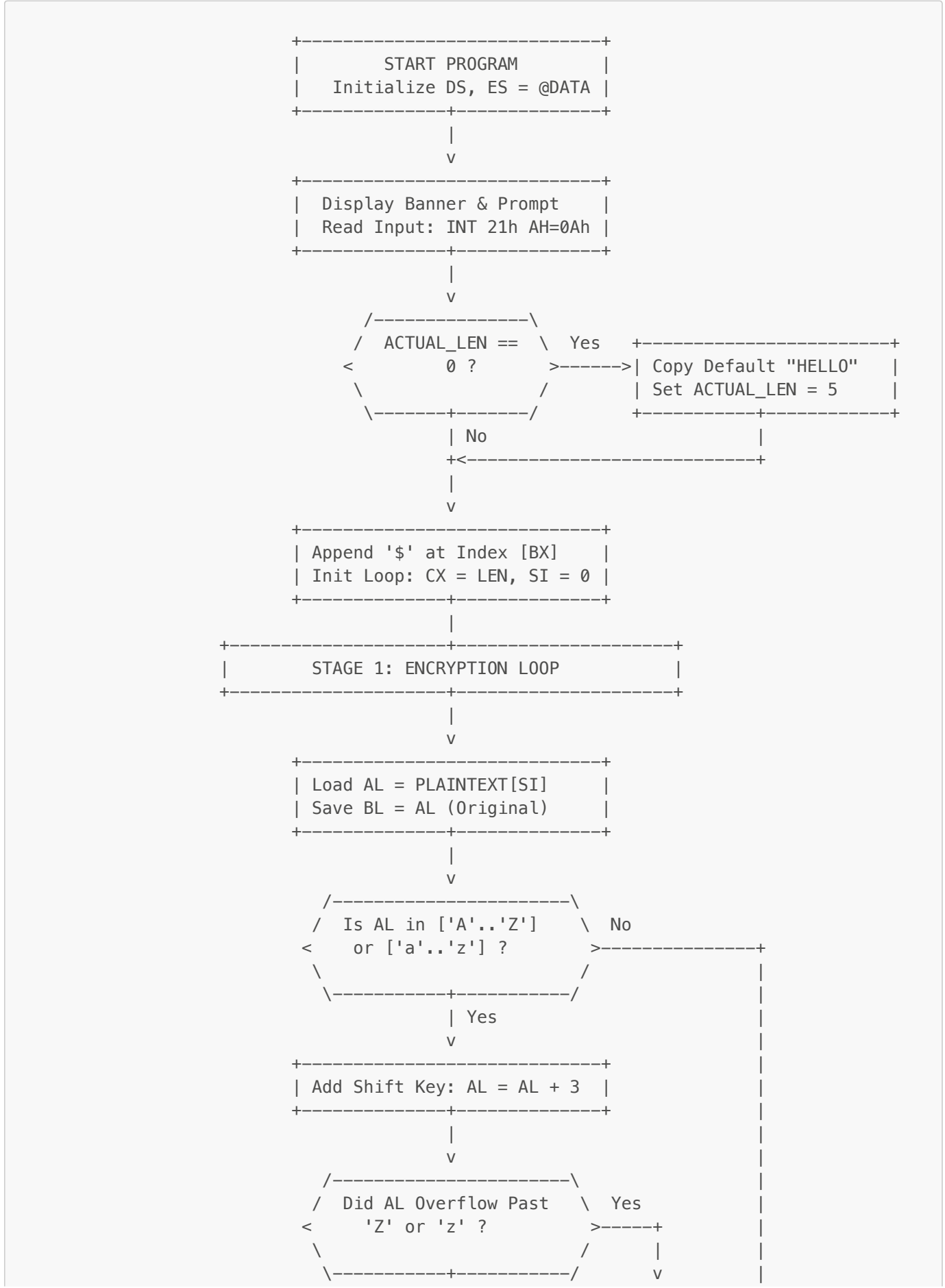


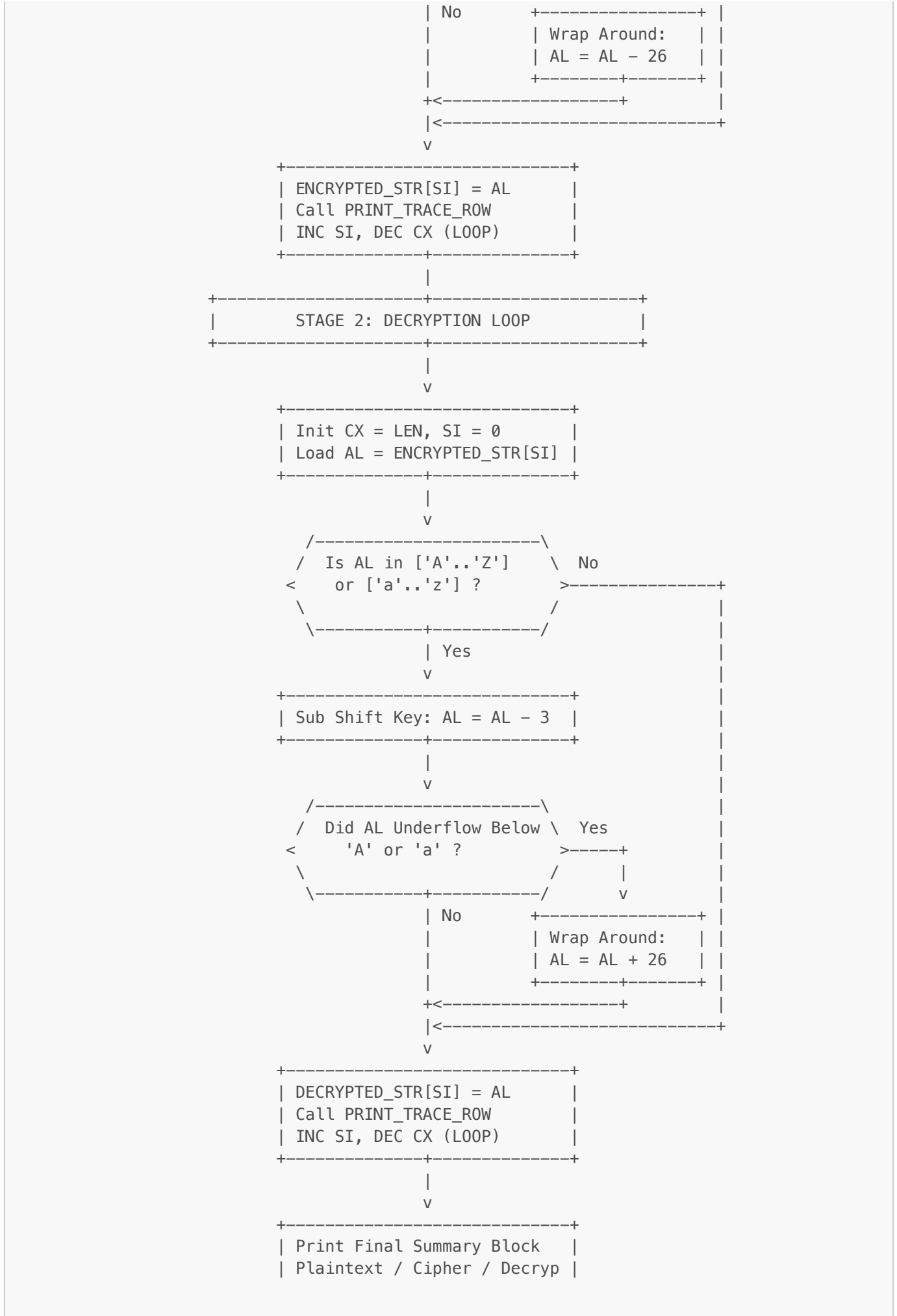
1. **Byte 0 (`INPUT_BUF`)**: Initialized by the programmer to `MAX_LEN` (`64`), specifying the maximum number of characters DOS is permitted to accept before beeping and ignoring further keystrokes.
2. **Byte 1 (`ACTUAL_LEN`)**: Populated automatically by MS-DOS upon the user pressing `Enter` (`0Dh`), recording the exact count of characters typed (excluding the trailing Carriage Return).
3. **Byte 2 Onward (`PLAINTEXT`)**: Contains the raw ASCII bytes entered by the user, terminated by `0Dh`. In `CAESAR.ASM`, the program uses `ACTUAL_LEN` in `BX` to overwrite the trailing `0Dh` with `'$'` (`24h`) so the buffer can be directly printed via DOS Function `09h`.

System Architecture and Algorithmic Design

End-to-End Execution Flowchart

The control flow of `CAESAR.ASM` is structured into five sequential stages: segment initialization, buffered user input acquisition (with automatic fallback to `"HELLO"` if the user presses `Enter` on an empty line), the forward encryption loop with live console visualization, the reverse decryption loop with live console visualization, and the final verification summary.




| No
| Wrap Around:
| AL = AL - 26
ENCRYPTED_STR[SI] = AL
Call PRINT_TRACE_ROW
INC SI, DEC CX (LOOP)
STAGE 2: DECRYPTION LOOP
Init CX = LEN, SI = 0
Load AL = ENCRYPTED_STR[SI]
Is AL in ['A'..'Z']
or ['a'..'z'] ?
No
Yes
Sub Shift Key: AL = AL - 3
Did AL Underflow Below
'A' or 'a' ?
Yes
No
Wrap Around:
AL = AL + 26
DECRYPTED_STR[SI] = AL
Call PRINT_TRACE_ROW
INC SI, DEC CX (LOOP)
Print Final Summary Block
Plaintext / Cipher / Decryp

```
                          | Exit: INT 21h, AH = 4Ch     |
                          +-----------------------------+
```

Detailed Analysis of Core Algorithmic Modules

Character Classification and Case-Preserving Branch Logic

A critical requirement in practical cipher implementations is distinguishing between uppercase characters, lowercase characters, and non-alphabetic characters (such as spaces, digits, or punctuation). In `CAESAR.ASM`, this classification is performed using unsigned conditional jump instructions (`JB`, `JBE`, `JA`, `JAE`) which evaluate the Carry Flag (`CF`) and Zero Flag (`ZF`) after a non-destructive `CMP` (Compare) instruction:

1. `CMP AL, 'A'` followed by `JB ENC_STORE`: If the unsigned ASCII byte in `AL` is strictly less than `41h` (`'A'`), `CF = 1`, and execution branches immediately to `ENC_STORE`, leaving spaces (`20h`), digits (`30h`–`39h`), and punctuation untouched.
2. `CMP AL, 'Z'` followed by `JBE ENC_UPPER`: Since `AL >= 'A'` is already guaranteed, if `AL <= 5Ah` (`'Z'`), `CF = 1` or `ZF = 1`, routing execution directly to the uppercase shift handler `ENC_UPPER`.
3. `CMP AL, 'a'` followed by `JB ENC_STORE`: Any character falling in the 6-byte ASCII gap (`5Bh`–`60h`) between `'Z'` and `'a'` triggers `CF = 1` and is preserved unchanged.
4. `CMP AL, 'z'` followed by `JA ENC_STORE`: Any character above `7Ah` (`'z'`) triggers `CF = 0` and `ZF = 0` and is preserved unchanged. Characters passing this final check are guaranteed to reside in `['a'..'z']` (`61h`–`7Ah`).

Hexadecimal Conversion via Hardware Table Lookup (`XLAT`)

To make the execution transparent for laboratory demonstration, `CAESAR.ASM` includes a dedicated `PRINT_HEX_BYTE` subroutine that displays the exact 8-bit hexadecimal ASCII value of every character before and after the $+3$ and -3 shifts. Rather than using slow conditional branches to test whether a 4-bit nibble is between `0`–`9` or `A`–`F`, the subroutine uses the 8086 hardware table-lookup instruction `XLAT` (Translate Byte):

- The upper nibble (bits 7–4) is isolated by shifting `AL` right by 4 bit positions (`SHR AL, 1` four times) and masking with `AND AL, 0Fh`.
- `BX` is loaded with the effective address of a 16-byte lookup table `HEX_DIGITS DB '0123456789ABCDEF'`.
- Executing `XLAT` replaces `AL` with the byte at memory address `DS:[BX + AL]` in a single instruction, immediately converting any value `00h`–`0Fh` into its printable ASCII hexadecimal digit.

Complete MASM Source Code Implementation (`CAESAR.ASM`)

The complete source code is stored in the connected laboratory directory at `/Users/sayanadityadas/academics/labs/masm/CAESAR.ASM` and is reproduced below with full inline documentation for laboratory review and viva examination.

```
; ===========================================================================
; PROGRAM     : CAESAR.ASM
; TITLE       : 8086 MASM Simulation of the Caesar Cipher (+3 Shift)
; ARCHITECTURE: Intel 8086 (16-Bit Real Mode, MS-DOS / DOSBox / EMU8086)
; ASSEMBLER   : Microsoft Macro Assembler (MASM) / TASM / EMU8086
; ===========================================================================
; DESCRIPTION :
;   This program implements the classical Caesar Cipher with a fixed shift key
;   of K = +3. It demonstrates:
;     a) Buffered user string input via DOS Interrupt 21h (Function 0Ah).
;     b) Character-by-character encryption: E(x) = (x + 3) mod 26
;        with full uppercase ('A'-'Z') and lowercase ('a'-'z') wrap-around
```

```
;        handling and preservation of non-alphabetic characters.
;     c) Step-by-step live console trace showing each character's original
;        ASCII code (in Hex), shifted ASCII code, and transformed symbol.
;     d) Character-by-character decryption: D(x) = (x - 3) mod 26
;        recovering the original plaintext string.
; ===========================================================================

.MODEL SMALL
.STACK 100H

.DATA
    ; ---------------------------------------------------------------------
    ; CONSTANTS & CONFIGURATION
    ; ---------------------------------------------------------------------
    SHIFT_KEY       EQU 3               ; Fixed Caesar shift key (+3)
    MAX_LEN         EQU 64              ; Maximum input string length

    ; ---------------------------------------------------------------------
    ; BUFFERS (DOS Function 0Ah Buffered Keyboard Input Structure)
    ; Byte 0: Maximum characters buffer can hold (including CR)
    ; Byte 1: Actual number of characters read by DOS (excluding CR)
    ; Byte 2+: The actual characters entered by the user
    ; ---------------------------------------------------------------------
    INPUT_BUF       DB  MAX_LEN
    ACTUAL_LEN      DB  0
    PLAINTEXT       DB  MAX_LEN DUP('$')

    ENCRYPTED_STR   DB  MAX_LEN DUP('$') ; Buffer for encrypted ciphertext
    DECRYPTED_STR   DB  MAX_LEN DUP('$') ; Buffer for decrypted plaintext

    ; ---------------------------------------------------------------------
    ; UI BANNER & STEP-BY-STEP VISUALIZATION STRINGS
    ; ---------------------------------------------------------------------
    BANNER_TOP      DB 13, 10,
'+============================================================+$'
    BANNER_TITLE    DB 13, 10, '|      8086 MASM SYSTEM LAB: CAESAR CIPHER SIMULATOR
(+3)      |$'
    BANNER_BOT      DB 13, 10,
'+============================================================+$'

    PROMPT_MSG      DB 13, 10, 13, 10, '[INPUT] Enter Plaintext String (e.g., HELLO):
$'
    DEFAULT_NOTE    DB 13, 10, '        (Empty input detected -> Using default:
"HELLO")$'
    DEFAULT_STR     DB 'HELLO'

    ; Step 1 Visualization Headers
    STEP1_HDR       DB 13, 10, 13, 10, '+-----------------------------------------
------------------+'
                    DB 13, 10, '| STEP 1: ENCRYPTION TRACE  [Formula: C = (P - Base +
3) % 26] |'
                    DB 13, 10, '+-------+-----------+-------------+---------------+--
----------+'
                    DB 13, 10, '| Index | Plain Chr | Plain ASCII | Shifted ASCII |
Cipher Chr |'
                    DB 13, 10, '+-------+-----------+-------------+---------------+--
----------+$'

    ; Step 2 Visualization Headers
    STEP2_HDR       DB 13, 10, 13, 10, '+-----------------------------------------
```

```
------------------+'
                    DB 13, 10, '| STEP 2: DECRYPTION TRACE  [Formula: P = (C - Base -
3) % 26] |'
                    DB 13, 10, '+-------+----------+------------+--------------+--
----------+'
                    DB 13, 10, '| Index | Ciphr Chr | Ciphr ASCII | Restord ASCII |
Plain Chr  |'
                    DB 13, 10, '+-------+----------+------------+--------------+--
----------+$'

    TABLE_SEP       DB 13, 10, '+-------+----------+------------+--------------+--
----------+$'

    ; Row Formatting Templates
    COL_BAR_1       DB 13, 10, '|   $'
    COL_BAR_2       DB '   |     $'
    COL_BAR_3       DB '     |     $'
    COL_BAR_4       DB 'h     |      $'
    COL_BAR_5       DB 'h      |     $'
    COL_BAR_END     DB '      |$'

    ; Final Summary Strings
    SUMMARY_HDR     DB 13, 10, 13, 10,
'+============================================================+'
                    DB 13, 10, '|                    FINAL CIPHER SUMMARY
|'
                    DB 13, 10,
'+============================================================+$'
    LBL_PLAIN       DB 13, 10, '  a) Original Plaintext  : $'
    LBL_ENCRYPTED   DB 13, 10, '  b) Encrypted (+3 Shift): $'
    LBL_DECRYPTED   DB 13, 10, '  c) Decrypted Plaintext : $'
    FOOTER_LINE     DB 13, 10,
'+============================================================+', 13, 10, '$'

    HEX_DIGITS      DB '0123456789ABCDEF' ; Lookup table for Hex visualization

.CODE
MAIN PROC
    ; -------------------------------------------------------------------------
    ; 1. INITIALIZE DATA SEGMENT
    ; -------------------------------------------------------------------------
    MOV AX, @DATA
    MOV DS, AX
    MOV ES, AX                          ; Initialize ES for string operations

    ; -------------------------------------------------------------------------
    ; 2. DISPLAY PROGRAM BANNER & ACCEPT USER INPUT (Requirement a)
    ; -------------------------------------------------------------------------
    LEA DX, BANNER_TOP
    CALL PRINT_STR
    LEA DX, BANNER_TITLE
    CALL PRINT_STR
    LEA DX, BANNER_BOT
    CALL PRINT_STR

    LEA DX, PROMPT_MSG
    CALL PRINT_STR

    ; Call DOS INT 21h, Function 0Ah (Buffered Keyboard Input)
    LEA DX, INPUT_BUF
```

```
    MOV AH, 0AH
    INT 21H

    ; Check if user pressed Enter without typing any characters
    MOV CL, ACTUAL_LEN
    CMP CL, 0
    JNE TERMINATE_INPUT

    ; Populate default "HELLO" if input is empty so simulation always works
    LEA DX, DEFAULT_NOTE
    CALL PRINT_STR
    MOV ACTUAL_LEN, 5
    LEA SI, DEFAULT_STR
    LEA DI, PLAINTEXT
    MOV CX, 5
    CLD
    REP MOVSB

TERMINATE_INPUT:
    ; Append '$' terminator to PLAINTEXT buffer for DOS Function 09h printing
    XOR BH, BH
    MOV BL, ACTUAL_LEN
    MOV PLAINTEXT[BX], '$'
    MOV ENCRYPTED_STR[BX], '$'
    MOV DECRYPTED_STR[BX], '$'

    ; ------------------------------------------------------------------------
    ; 3. ENCRYPT PLAINTEXT USING CAESAR SHIFT OF +3 (Requirement b)
    ; ------------------------------------------------------------------------
    LEA DX, STEP1_HDR
    CALL PRINT_STR

    XOR CX, CX
    MOV CL, ACTUAL_LEN                  ; CX = Loop counter (number of chars)
    XOR SI, SI                          ; SI = Current character index (0..N-1)

ENCRYPT_LOOP:
    MOV AL, PLAINTEXT[SI]               ; Load plaintext byte into AL
    MOV BL, AL                          ; Save original byte in BL for trace

    ; Check if character is Uppercase ('A' = 41h to 'Z' = 5Ah)
    CMP AL, 'A'
    JB  ENC_STORE                       ; Below 'A' -> Non-alphabetic, skip shift
    CMP AL, 'Z'
    JBE ENC_UPPER                       ; Between 'A' and 'Z' -> Uppercase shift

    ; Check if character is Lowercase ('a' = 61h to 'z' = 7Ah)
    CMP AL, 'a'
    JB  ENC_STORE                       ; Between 'Z' and 'a' -> Non-alphabetic
    CMP AL, 'z'
    JA  ENC_STORE                       ; Above 'z' -> Non-alphabetic

ENC_LOWER:
    ADD AL, SHIFT_KEY                   ; Apply +3 shift: AL = AL + 3
    CMP AL, 'z'                         ; Did it overflow past 'z' (7Ah)?
    JBE ENC_STORE
    SUB AL, 26                          ; Wrap around alphabet: AL = AL - 26
    JMP ENC_STORE

ENC_UPPER:
```

```
    ADD AL, SHIFT_KEY                   ; Apply +3 shift: AL = AL + 3
    CMP AL, 'Z'                         ; Did it overflow past 'Z' (5Ah)?
    JBE ENC_STORE
    SUB AL, 26                          ; Wrap around alphabet: AL = AL - 26

ENC_STORE:
    MOV ENCRYPTED_STR[SI], AL           ; Store encrypted byte into ciphertext buffer
    MOV BH, AL                          ; BH = Transformed character for trace

    ; Print live step-by-step visualization row for this character
    CALL PRINT_TRACE_ROW

    INC SI                              ; Advance to next character index
    LOOP ENCRYPT_LOOP

    LEA DX, TABLE_SEP
    CALL PRINT_STR

    ; ------------------------------------------------------------------------
    ; 4. DECRYPT CIPHERTEXT BACK TO ORIGINAL PLAINTEXT (Requirement c)
    ; ------------------------------------------------------------------------
    LEA DX, STEP2_HDR
    CALL PRINT_STR

    XOR CX, CX
    MOV CL, ACTUAL_LEN                  ; Reload string length into CX
    XOR SI, SI                          ; Reset index SI to 0

DECRYPT_LOOP:
    MOV AL, ENCRYPTED_STR[SI]           ; Load encrypted character into AL
    MOV BL, AL                          ; Save input ciphertext char in BL

    ; Check if character is Uppercase ('A' to 'Z')
    CMP AL, 'A'
    JB  DEC_STORE
    CMP AL, 'Z'
    JBE DEC_UPPER

    ; Check if character is Lowercase ('a' to 'z')
    CMP AL, 'a'
    JB  DEC_STORE
    CMP AL, 'z'
    JA  DEC_STORE

DEC_LOWER:
    SUB AL, SHIFT_KEY                   ; Apply -3 reverse shift: AL = AL - 3
    CMP AL, 'a'                         ; Did it underflow below 'a' (61h)?
    JAE DEC_STORE
    ADD AL, 26                          ; Wrap around alphabet: AL = AL + 26
    JMP DEC_STORE

DEC_UPPER:
    SUB AL, SHIFT_KEY                   ; Apply -3 reverse shift: AL = AL - 3
    CMP AL, 'A'                         ; Did it underflow below 'A' (41h)?
    JAE DEC_STORE
    ADD AL, 26                          ; Wrap around alphabet: AL = AL + 26

DEC_STORE:
    MOV DECRYPTED_STR[SI], AL           ; Store recovered plaintext byte
    MOV BH, AL                          ; BH = Restored character for trace
```

```
    ; Print live step-by-step visualization row for this character
    CALL PRINT_TRACE_ROW

    INC SI                          ; Advance index
    LOOP DECRYPT_LOOP

    LEA DX, TABLE_SEP
    CALL PRINT_STR

    ; ------------------------------------------------------------------------
    ; 5. DISPLAY FINAL SUMMARY OF RESULTS
    ; ------------------------------------------------------------------------
    LEA DX, SUMMARY_HDR
    CALL PRINT_STR

    LEA DX, LBL_PLAIN
    CALL PRINT_STR
    LEA DX, PLAINTEXT
    CALL PRINT_STR

    LEA DX, LBL_ENCRYPTED
    CALL PRINT_STR
    LEA DX, ENCRYPTED_STR
    CALL PRINT_STR

    LEA DX, LBL_DECRYPTED
    CALL PRINT_STR
    LEA DX, DECRYPTED_STR
    CALL PRINT_STR

    LEA DX, FOOTER_LINE
    CALL PRINT_STR

    ; ------------------------------------------------------------------------
    ; 6. TERMINATE PROGRAM CLEANLY (DOS INT 21h, Function 4Ch)
    ; ------------------------------------------------------------------------
    MOV AH, 4CH
    XOR AL, AL                      ; Return code 0
    INT 21H
MAIN ENDP

; ============================================================================
; SUBROUTINE: PRINT_STR
; Purpose   : Prints a '$'-terminated string pointed to by DS:DX
; Inputs    : DX = Offset of string in Data Segment
; Preserves : AX, BX, CX, SI
; ============================================================================
PRINT_STR PROC
    PUSH AX
    MOV AH, 09H
    INT 21H
    POP AX
    RET
PRINT_STR ENDP

; ============================================================================
; SUBROUTINE: PRINT_CHAR
; Purpose   : Prints a single ASCII character in DL to standard output
; Inputs    : DL = Character to print
```

```
; Preserves : AX, BX, CX, DX, SI
; ================================================================================
PRINT_CHAR PROC
    PUSH AX
    MOV AH, 02H
    INT 21H
    POP AX
    RET
PRINT_CHAR ENDP

; ================================================================================
; SUBROUTINE: PRINT_HEX_BYTE
; Purpose   : Prints an 8-bit value in AL as a 2-digit hexadecimal number
; Inputs    : AL = Byte value to display in Hex
; Preserves : AX, BX, CX, DX, SI
; ================================================================================
PRINT_HEX_BYTE PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    MOV AH, AL                          ; Save full byte in AH

    ; Extract upper nibble (bits 7..4)
    SHR AL, 1
    SHR AL, 1
    SHR AL, 1
    SHR AL, 1
    AND AL, 0FH
    LEA BX, HEX_DIGITS
    XLAT                                ; AL = HEX_DIGITS[AL]
    MOV DL, AL
    CALL PRINT_CHAR

    ; Extract lower nibble (bits 3..0)
    MOV AL, AH
    AND AL, 0FH
    LEA BX, HEX_DIGITS
    XLAT                                ; AL = HEX_DIGITS[AL]
    MOV DL, AL
    CALL PRINT_CHAR

    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_HEX_BYTE ENDP

; ================================================================================
; SUBROUTINE: PRINT_TRACE_ROW
; Purpose   : Prints a formatted step-by-step table row showing:
;             | Index | Input Char | Input ASCII (Hex) | Output ASCII | Output Char |
; Inputs    : SI = Current 0-based index
;             BL = Input character (Plaintext in Step 1, Ciphertext in Step 2)
;             BH = Output character (Ciphertext in Step 1, Plaintext in Step 2)
; Preserves : All registers (AX, BX, CX, DX, SI)
; ================================================================================
PRINT_TRACE_ROW PROC
```

```
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    ; Column 1: Index (single digit 0-9 or low digit of index)
    LEA DX, COL_BAR_1
    CALL PRINT_STR
    MOV AX, SI
    XOR DX, DX
    MOV CX, 10
    DIV CX                          ; DL = SI mod 10
    ADD DL, '0'
    CALL PRINT_CHAR

    ; Column 2: Input Character
    LEA DX, COL_BAR_2
    CALL PRINT_STR
    MOV DL, BL
    CALL PRINT_CHAR

    ; Column 3: Input ASCII Hex Value
    LEA DX, COL_BAR_3
    CALL PRINT_STR
    MOV AL, BL
    CALL PRINT_HEX_BYTE

    ; Column 4: Output ASCII Hex Value
    LEA DX, COL_BAR_4
    CALL PRINT_STR
    MOV AL, BH
    CALL PRINT_HEX_BYTE

    ; Column 5: Output Character
    LEA DX, COL_BAR_5
    CALL PRINT_STR
    MOV DL, BH
    CALL PRINT_CHAR

    LEA DX, COL_BAR_END
    CALL PRINT_STR

    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_TRACE_ROW ENDP

END MAIN
```

Step-by-Step Execution Trace and Register-Level Walkthrough

Trace 1: Canonical Benchmark (`Plaintext = "HELLO"`, Shift $= +3$)

When the user enters `HELLO` at the prompt, `INT 21h / AH=0Ah` stores `05h` in `ACTUAL_LEN` and populates `PLAINTEXT[0..4]` with `'H'`, `'E'`, `'L'`, `'L'`, `'O'`.

Forward Encryption Trace (`ENCRYPT_LOOP`)

Iteration (`SI`)	Loop Counter (`CX` Start → End)	Input Char (`BL`)	Input ASCII (Hex / Dec)	Zero-Based Ordinal (P)	ALU Operation (`ADD AL, 3`)	Boundary Check (`CMP AL, 5Ah`)	Wrap-Around Applied?	Output ASCII (Hex / Dec)	Cipher Char (`BH`)
`0000h`	`0005h` → `0004h`	`'H'`	`48h` (72_{10})	7	`48h + 03h = 4Bh`	`4Bh <= 5Ah` (`CF=0, ZF=0`)	No (`JBE` taken)	`4Bh` (75_{10})	`'K'`
`0001h`	`0004h` → `0003h`	`'E'`	`45h` (69_{10})	4	`45h + 03h = 48h`	`48h <= 5Ah` (`CF=0, ZF=0`)	No (`JBE` taken)	`48h` (72_{10})	`'H'`
`0002h`	`0003h` → `0002h`	`'L'`	`4Ch` (76_{10})	11	`4Ch + 03h = 4Fh`	`4Fh <= 5Ah` (`CF=0, ZF=0`)	No (`JBE` taken)	`4Fh` (79_{10})	`'O'`
`0003h`	`0002h` → `0001h`	`'L'`	`4Ch` (76_{10})	11	`4Ch + 03h = 4Fh`	`4Fh <= 5Ah` (`CF=0, ZF=0`)	No (`JBE` taken)	`4Fh` (79_{10})	`'O'`
`0004h`	`0001h` → `0000h`	`'O'`	`4Fh` (79_{10})	14	`4Fh + 03h = 52h`	`52h <= 5Ah` (`CF=0, ZF=0`)	No (`JBE` taken)	`52h` (82_{10})	`'R'`

Reverse Decryption Trace (`DECRYPT_LOOP`)

Iteration (`SI`)	Loop Counter (`CX` Start → End)	Cipher Char (`BL`)	Cipher ASCII (Hex / Dec)	Zero-Based Ordinal (C)	ALU Operation (`SUB AL, 3`)	Boundary Check (`CMP AL, 41h`)	Wrap-Around Applied?	Restored ASCII (Hex / Dec)	Decrypted Char (`BH`)
`0000h`	`0005h` → `0004h`	`'K'`	`4Bh` (75_{10})	10	`4Bh - 03h = 48h`	`48h >= 41h` (`CF=0`)	No (`JAE` taken)	`48h` (72_{10})	`'H'`
`0001h`	`0004h` → `0003h`	`'H'`	`48h` (72_{10})	7	`48h - 03h = 45h`	`45h >= 41h` (`CF=0`)	No (`JAE` taken)	`45h` (69_{10})	`'E'`
`0002h`	`0003h` → `0002h`	`'O'`	`4Fh` (79_{10})	14	`4Fh - 03h = 4Ch`	`4Ch >= 41h` (`CF=0`)	No (`JAE` taken)	`4Ch` (76_{10})	`'L'`
`0003h`	`0002h` → `0001h`	`'O'`	`4Fh` (79_{10})	14	`4Fh - 03h = 4Ch`	`4Ch >= 41h` (`CF=0`)	No (`JAE` taken)	`4Ch` (76_{10})	`'L'`

Iteration (SI)	Loop Counter (CX Start → End)	Cipher Char (BL)	Cipher ASCII (Hex / Dec)	Zero-Based Ordinal (C)	ALU Operation (SUB AL, 3)	Boundary Check (CMP AL, 41h)	Wrap-Around Applied?	Restored ASCII (Hex / Dec)	Decrypted Char (BH)
0004h	0001h → 0000h	'R'	52h (82_{10})	17	52h − 03h = 4Fh	4Fh >= 41h (CF=0)	No (JAE taken)	4Fh (79_{10})	'O'

Trace 2: Boundary Wrap-Around and Non-Alphabetic Stress Test ("XYZ abc!")

To verify that modular wrap-around (SUB AL, 26 during encryption and ADD AL, 26 during decryption) and non-alphabetic preservation operate accurately, consider the edge-case input "XYZ abc!":

Index (SI)	Input Char	Input ASCII	Encryption ALU Step	Wrap Adjustment (Encryption)	Encrypted Char	Decryption ALU Step	Wrap Adjustment (Decryption)	Restored Char
0	'X'	58h (88)	58h + 03h = 5Bh (> 'Z')	5Bh − 1Ah = 41h (65)	'A' (41h)	41h − 03h = 3Eh (< 'A')	3Eh + 1Ah = 58h (88)	'X' (58h)
1	'Y'	59h (89)	59h + 03h = 5Ch (> 'Z')	5Ch − 1Ah = 42h (66)	'B' (42h)	42h − 03h = 3Fh (< 'A')	3Fh + 1Ah = 59h (89)	'Y' (59h)
2	'Z'	5Ah (90)	5Ah + 03h = 5Dh (> 'Z')	5Dh − 1Ah = 43h (67)	'C' (43h)	43h − 03h = 40h (< 'A')	40h + 1Ah = 5Ah (90)	'Z' (5Ah)
3	' '	20h (32)	Bypassed (< 'A')	None (20h preserved)	' ' (20h)	Bypassed (< 'A')	None (20h preserved)	' ' (20h)
4	'a'	61h (97)	61h + 03h = 64h (<= 'z')	None (64h within range)	'd' (64h)	64h − 03h = 61h (>= 'a')	None (61h within range)	'a' (61h)
5	'b'	62h (98)	62h + 03h = 65h (<= 'z')	None (65h within range)	'e' (65h)	65h − 03h = 62h (>= 'a')	None (62h within range)	'b' (62h)
6	'c'	63h (99)	63h + 03h = 66h (<= 'z')	None (66h within range)	'f' (66h)	66h − 03h = 63h (>= 'a')	None (63h within range)	'c' (63h)
7	'!'	21h (33)	Bypassed (< 'A')	None (21h preserved)	'!' (21h)	Bypassed (< 'A')	None (21h preserved)	'!' (21h)

Simulated Console Output and Memory Dump Visualization

Console Output Transcript for Input "HELLO"

When CAESAR.EXE is assembled and executed in DOSBox or EMU8086 with the user input HELLO, the program renders the following formatted console output:

```
+============================================================+
|      8086 MASM SYSTEM LAB: CAESAR CIPHER SIMULATOR (+3)    |
+============================================================+

[INPUT] Enter Plaintext String (e.g., HELLO): HELLO

+------------------------------------------------------------+
| STEP 1: ENCRYPTION TRACE  [Formula: C = (P - Base + 3) % 26] |
+-------+-----------+-------------+---------------+------------+
| Index | Plain Chr | Plain ASCII | Shifted ASCII | Cipher Chr |
+-------+-----------+-------------+---------------+------------+
|   0   |     H     |     48h     |      4Bh      |     K      |
|   1   |     E     |     45h     |      48h      |     H      |
|   2   |     L     |     4Ch     |      4Fh      |     O      |
|   3   |     L     |     4Ch     |      4Fh      |     O      |
|   4   |     O     |     4Fh     |      52h      |     R      |
+-------+-----------+-------------+---------------+------------+

+------------------------------------------------------------+
| STEP 2: DECRYPTION TRACE  [Formula: P = (C - Base - 3) % 26] |
+-------+-----------+-------------+---------------+------------+
| Index | Ciphr Chr | Ciphr ASCII | Restord ASCII | Plain Chr  |
+-------+-----------+-------------+---------------+------------+
|   0   |     K     |     4Bh     |      48h      |     H      |
|   1   |     H     |     48h     |      45h      |     E      |
|   2   |     O     |     4Fh     |      4Ch      |     L      |
|   3   |     O     |     4Fh     |      4Ch      |     L      |
|   4   |     R     |     52h     |      4Fh      |     O      |
+-------+-----------+-------------+---------------+------------+

+============================================================+
|                    FINAL CIPHER SUMMARY                    |
+============================================================+
  a) Original Plaintext  : HELLO
  b) Encrypted (+3 Shift): KHOOR
  c) Decrypted Plaintext : HELLO
+============================================================+
```

Data Segment (DS) Hexadecimal Memory Dump Visualization

Inspecting the Data Segment (DS:0000h) after Step 4 (Decryption) in DEBUG.EXE or the EMU8086 Memory Viewer reveals how the three buffers (INPUT_BUF/PLAINTEXT, ENCRYPTED_STR, and DECRYPTED_STR) reside contiguously in RAM:

```
+-------------------------------------------------------------------------+
|               DATA SEGMENT (DS) MEMORY DUMP AFTER EXECUTION             |
+-------------------------------------------------------------------------+
|                                                                         |
|  DS:0000h  [INPUT_BUF]   : 40h (64 Max Length)                          |
|  DS:0001h  [ACTUAL_LEN]  : 05h (5 Characters Read)                      |
|                                                                         |
|  DS:0002h  [PLAINTEXT]   : 48h  45h  4Ch  4Ch  4Fh  24h  24h  24h ...  |
|  ASCII Representation    : 'H'  'E'  'L'  'L'  'O'  '$'  '$'  '$'       |
|                            |    |    |    |    |                       |
|                            |    |    |    |    +---> +3 Shift (4Fh -> 52h) |
|                            +----+----+----+--------> +3 Shift           |
```

```
|                              v    v    v    v    v                          |
|  DS:0042h  [ENCRYPTED]    : 4Bh  48h  4Fh  4Fh  52h  24h  24h  24h ...      |
|  ASCII Representation     : 'K'  'H'  'O'  'O'  'R'  '$'  '$'  '$'          |
|                              |    |    |    |    |                          |
|                              |    |    |    |    +---> -3 Shift (52h -> 4Fh) |
|                              +----+----+----+--------> -3 Shift              |
|                              v    v    v    v    v                          |
|  DS:0082h  [DECRYPTED]    : 48h  45h  4Ch  4Ch  4Fh  24h  24h  24h ...      |
|  ASCII Representation     : 'H'  'E'  'L'  'L'  'O'  '$'  '$'  '$'          |
+-------------------------------------------------------------------------+
```

Build, Assembly, and Execution Instructions

The source file `/Users/sayanadityadas/academics/labs/masm/CAESAR.ASM` is compatible with standard 16-bit x86 toolchains including `MASM 5.0/6.11` under `DOSBox`, `Turbo Assembler (TASM)`, and `EMU8086`.

Method 1: Running in DOSBox with MASM and LINK

1. Mount your local MASM directory inside `DOSBox`:

   ```
   MOUNT C /Users/sayanadityadas/academics/labs/masm
   C:
   ```

2. Assemble `CAESAR.ASM` into an object file (`CAESAR.OBJ`):

   ```
   MASM CAESAR.ASM;
   ```

3. Link the object file to generate the executable (`CAESAR.EXE`):

   ```
   LINK CAESAR.OBJ;
   ```

4. Execute the simulator:

   ```
   CAESAR.EXE
   ```

Method 2: Running in EMU8086 (Step-by-Step Register Visualization)

1. Open `EMU8086` and load `/Users/sayanadityadas/academics/labs/masm/CAESAR.ASM`.
2. Click **Emulate** to open the dual-pane register/memory debugger and the virtual console window.
3. Click **Single Step** to watch `AL`, `BL`, `BH`, `SI`, and `CX` update on each character iteration, or click **Run** to interact with the console prompt directly.

Complexity Analysis, Cryptanalysis, and Architectural Conclusions

Computational and Space Complexity on the Intel 8086

Let N denote the length of the input string stored in `ACTUAL_LEN` ($0 \leq N \leq 64$).

- **Time Complexity**: Both `ENCRYPT_LOOP` and `DECRYPT_LOOP` execute exactly N iterations. Inside each iteration, character classification requires at most 4 comparisons (`CMP`) and 4 conditional jumps, followed by 1 addition or subtraction (`ADD`/`SUB` taking 3–4 clock cycles on the 8086) and at most 1 wrap-around adjustment (`SUB`/`ADD 26`). Avoiding the 8086 hardware `DIV` instruction (which requires 80–90 clock cycles per byte) reduces the core cryptographic transform to fewer than 35 clock cycles per character—an $\mathcal{O}(N)$ linear-time execution profile with minimal constant overhead.
- **Space Complexity**: The program allocates three static 64-byte buffers (`PLAINTEXT`, `ENCRYPTED_STR`, `DECRYPTED_STR`) in the `.DATA` segment alongside constant UI strings, yielding a fixed $\mathcal{O}(1)$ auxiliary memory footprint under 1.5 KB.

Security Evaluation and Classical Cryptanalysis

While the Caesar Cipher serves as an instructive model for modular arithmetic and low-level register manipulation, it provides zero computational security against modern or classical cryptanalysis (Caesar cipher - Wikipedia):

1. **Exhaustive Key Space Search (Brute-Force Attack)**: Because the key space is restricted to 25 non-trivial shifts ($K \in \{1, \dots, 25\}$), an adversary can test all candidate keys in microseconds.
2. **Monoalphabetic Frequency Analysis**: Because each plaintext letter maps bijectively to a single ciphertext letter, the statistical frequency distribution of the underlying natural language (for example, `'E'`, `'T'`, `'A'`, `'O'`, `'I'`, `'N'` in English text) is preserved in the ciphertext, shifted solely by a constant offset of $+3$. Computing the chi-squared (χ^2) statistic or single-letter histogram immediately reveals $K = 3$ without requiring knowledge of the key.

References

1. Wikipedia Contributors, "Caesar cipher," *Wikipedia, The Free Encyclopedia*.
2. European Information Technologies Certification Academy (EITCA), "How does the Caesar Cipher utilize the concept of modular arithmetic to encrypt and decrypt messages?," *EITC/IS/CCF Classical Cryptography Fundamentals*, 2024.
3. Splunk Security Education, "The Caesar Cipher, Explained," *Splunk Blog*, 2024.
4. Yassine Bridi, "Basic 8086 and DOS Interrupts Reference (INT 21h / AH=0Ah Buffered Input)," *8086 BIOS and DOS Interrupts Documentation*.
5. Stack Overflow Community, "Understanding Interrupt 21h, AH=0Ah Buffered String Input in 8086 Assembly," *Stack Overflow*.
6. Wikipedia Contributors, "Intel 8086 Microprocessor Architecture," *Wikipedia, The Free Encyclopedia*.
7. Wikipedia Contributors, "Microsoft Macro Assembler (MASM)," *Wikipedia, The Free Encyclopedia*.
8. Wikipedia Contributors, "ASCII Character Encoding Standard," *Wikipedia, The Free Encyclopedia*.